



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

*«Загружаемый модуль ядра, позволяющий скрывать
файлы или запрещать их изменение, чтение и
удаление»*

Студент ИУ7-72Б
(Группа)

(Подпись, дата)

А. А. Новиков
(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

Н. Ю. Рязанова
(И. О. Фамилия)

2025 г.

Задание

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Аналитический раздел	6
1.1 Способы перехвата функций ядра	6
1.1.1 ftrace	6
1.1.2 kprobes	7
1.1.3 Linux Security API	8
1.1.4 Модификация таблицы системных вызовов	8
1.1.5 Сплайсинг	9
1.1.6 Сравнительный анализ способов перехвата	9
1.2 Перехватываемые функции ядра	10
1.2.1 Функция getdents64	10
1.3 Вывод	12
2 Конструкторский раздел	13
2.1 IDEF0	13
2.2 Алгоритм проверки необходимости сокрытия файла	14
2.3 Алгоритм проверки разрешения на удаление файла	14
2.4 Алгоритм проверки разрешения на запись в файл	15
2.5 Алгоритм проверки разрешения на чтение из файла	16
3 Технологический раздел	17
3.1 Выбор языка и среды программирования	17
3.2 Реализация алгоритма проверки необходимости сокрытия файла	17
3.3 Реализация алгоритма проверки разрешения на удаление файла	18
3.4 Реализация алгоритма проверки разрешения на чтение из файла	19
3.5 Реализация алгоритма проверки разрешения на запись в файл .	20
3.6 Инициализация полей структуры ftrace_hook	20
3.7 Makefile	21
ЗАКЛЮЧЕНИЕ	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24

ВВЕДЕНИЕ

Обеспечение безопасного доступа к файлам в операционной системе Linux является актуальной задачей. При работе с файлами в Linux необходимо обеспечивать конфиденциальность данных и предоставлять защиту от вредоносных действий.

Целью данной курсовой работы является разработка загружаемого модуля ядра для ОС Linux, позволяющего скрывать файлы или запрещать их изменение, чтение и удаление. Необходимо предусмотреть возможность ввода пароля для отображения файлов или разрешения операций над ними. Предоставить пользователю возможность задавать список таких файлов.

Для решения поставленной задачи необходимо:

- проанализировать возможности перехвата функций ядра Linux;
- выбрать системные вызовы, которые необходимо перехватить;
- разработать алгоритм перехвата, алгоритмы hook-функций и структуру программного обеспечения;
- реализовать программное обеспечение;
- исследовать работу ПО.

1 Аналитический раздел

1.1 Способы перехвата функций ядра

1.1.1 ftrace

ftrace предоставляет возможности для трассировки функций. С его помощью можно отслеживать контекстные переключения, измерять время обработки прерываний, высчитывать время на активизацию заданий с высоким приоритетом и многое другое [1].

Ftrace был разработан Стивеном Росгедтом и добавлен в ядро в 2008 году, начиная с версии 2.6.27. Ftrace — фреймворк, предоставляющий отладочный кольцевой буфер для записи данных. Собирают эти данные встроенные в ядро программы-трассировщики [1].

Работает ftrace на базе файловой системы debugfs, которая в большинстве современных дистрибутивов Linux смонтирована по умолчанию.

Каждую перехватываемую функцию можно описать следующей структурой:

Листинг 1.1 – Структура ftrace_book

```
1 struct ftrace_book {  
2     const char *name;  
3     void *function;  
4     void *original;  
5  
6     unsigned long address;  
7     struct ftrace_ops ops;  
8 };
```

Поля структуры:

- name — имя перехватываемой функции;
- function — адрес функции-сборки, которая будет вызываться вместо перехваченной функции;
- original — указатель на место, куда следует записать адрес перехватываемой функции, заполняется при установке;
- address — адрес перехватываемой функции, заполняется при установке;

— ops — служебная информация ftrace.

Листинг 1.2 – Пример заполнения структуры ftrace_book

```
1 #define HOOK(_name, _function, _original) \  
2 { \  
3     .name = (_name), \  
4     .function = (_function), \  
5     .original = (_original), \  
6 } \  
7 \  
8 static struct ftrace_book hooked_functions[] = { \  
9     HOOK("sys_clone", fh_sys_clone, kreal_sys_clone), \  
10    HOOK("sys_execve", fh_sys_execve, kreal_sys_execve), \  
11 };
```

1.1.2 kprobes

Kprobes — специализированное API, в первую очередь предназначенное для отладки и трассирования ядра. Этот интерфейс позволяет устанавливать пред- и постобработчики для любой инструкции в ядре, а также обработчики на вход и возврат из функции. Обработчики получают доступ к регистрам и могут их изменять.

K probes реализуются с помощью точек останова (инструкции `int3`), внедряемых в исполнимый код ядра, что позволяет устанавливать k probes в любом месте любой функции, если оно известно. Аналогично, kretprobes реализуются через подмену адреса возврата на стеке и позволяют перехватить возврат из любой функции.

При использовании k probes для получения аргументов функции или значений локальных переменных надо знать, в каких регистрах или где на стеке они лежат, и самостоятельно их оттуда извлекать. Для решения данной проблемы существует j probes — надстройка над k probes, самостоятельно извлекающая аргументы функции из регистров или стека и вызывающая обработчик, который должен иметь ту же сигнатуру, что и перехватываемая функция. Однако j probes объявлен устаревшим и удален из современных ядер.

1.1.3 Linux Security API

Linux Security API — интерфейс, созданный для перехвата функций ядра. В критических местах кода ядра расположены вызовы security-функций, которые в свою очередь вызывают коллэски, установленные security-модулем. Security-модуль может анализировать контекст операции и принимать решение о ее разрешении или запрете.

Для Linux Security API характерны следующие ограничения:

- security-модули не могут быть загружены динамически, они являются частью ядра и требуют его перекомпиляции;
- в системе может быть только один security-модуль.

Таким образом, для использования Security API необходимо поставлять собственную сборку ядра, а также интегрировать дополнительный модуль с SELinux или AppArmor, которые используются популярными дистрибутивами.

1.1.4 Модификация таблицы системных вызовов

В ядре Linux все обработчики системных вызовов хранятся в таблице `sys_call_table` [2]. Подмена значений в этой таблице приводит к смене поведения всей системы. Таким образом, сохранив старое значение обработчика и подставить в таблицу собственный обработчик, можно перехватить системный вызов.

Однако данный подход обладает следующими недостатками:

- Техническая сложность реализации, заключающаяся в необходимости обхода защиты от модификации таблицы, атомарное и безопасное выполнение замены.
- Невозможность перехвата некоторых обработчиков. В ядрах до версии 4.16 обработка системных вызовов для архитектуры `x86_64` содержала целый ряд оптимизаций. Некоторые из них требовали того, что обработчик системного вызова являлся специальным переходником, реализованным на ассемблере. Соответственно, подобные обработчики порой сложно, а иногда и вовсе невозможно заменить на собственные, написанные на Си [3].

1.1.5 Сплайсинг

Сплайсинг заключается в замене инструкций в начале функции на безусловный переход, ведущий в обработчик. Оригинальные инструкции переносятся в другое место и исполняются перед переходом обратно в перехваченную функцию. Именно таким образом реализуется jump-оптимизация для kprobes. Используя сплайсинг, можно добиться тех же результатов, но без дополнительных расходов на kprobes и с полным контролем ситуации.

Сложность использования сплайсинга заключается в необходимости синхронизации установки и снятия перехвата, обхода защиты от модификации регионов памяти с кодом, инвалидации коней процессора после замены инструкций, дизассемблирования заменяемых инструкций и проверки на отсутствие переходов внутрь заменяемого кода.

1.1.6 Сравнительный анализ способов перехвата

Результаты сравнения различных способов перехвата функций ядра представлены в таблице 1.1

Таблица 1.1 – Результаты сравнения

Способ перехвата	Необходимость перекомпиляции ядра	Возможность перехвата любых обработчиков	Доступ к аргументам функции через переменные	Необходимость обхода защиты от модификации регионов памяти
Linux Security API	+	—	+	—
Модификация таблиц системных вызовов	—	—	+	+
kprobes	—	+	—	—
Сплайсинг	—	+	+	+
ftrace	—	+	+	—

1.2 Перехватываемые функции ядра

1.2.1 Функция getdents64

Для сокрытия файла необходимо перехватить функцию ядра `getdents64`, так как она возвращает записи каталога.

Листинг 1.3 – Функция `getdents64`

```
1 #include <fcntl.h>
2
3 int getdents64(unsigned int fd, struct linux_dirent64 *dirp,
    unsigned int count);
```

Системный вызов `getdents64` читает несколько структур `linux_dirent64` из каталога, на который указывает открытый файловый дескриптор `fd`, в буфер, указанный в `dirp`. В аргументе `count` задается размер этого буфера.

Структура `linux_dirent64` определена следующим образом:

Листинг 1.4 – Структура `linux_dirent64`

```
1 struct linux_dirent64 {
2     ino64_t    d_ino;
3     off64_t    d_off;
4     unsigned short d_reclen;
5     unsigned char  d_type;
6     char       d_name[];
7 };
```

В `d_ino` указан номер `inode`. В `d_off` задается расстояние от начала каталога до начала следующей `linux_dirent64`. В `d_reclen` указывается размер данной `linux_dirent64`. В `d_name` задается имя файла, в `d_type` — тип файла.

Таким образом, перехват системного вызова `getdents64` позволяет удалить запись из списка записей каталога. Перехват вызова осуществляется при помощи создания указателя на системный вызов `getdents64`.

Листинг 1.5 – Указатель на `getdents64`

```
1 static asmlinkage long (*real_sys_getdents64)(const struct
    pt_regs *);
```

Функция `unlink`

Удаление записей из каталога производится с помощью функции `unlink`.

Листинг 1.6 – Функция unlink

```
1 #include <unistd.h>
2
3 int unlink(const char *pathname);
```

Эта функция удаляет запись из файла каталога и уменьшает значение счетчика ссылок на файл `pathname`. Если на файл указывает несколько ссылок, то его содержимое будет через них по-прежнему доступно.

Таким образом, для того, чтобы запретить удаление файла, необходимо перехватить системный вызов `unlink`. Перехват вызова осуществляется при помощи создания указателя на системный вызов.

Листинг 1.7 – Указатель на unlink

```
1 static asmlinkage long (*real_sys_unlinkat)(struct pt_regs *regs);
```

Функции open, write, read

Запись в файл осуществляется при помощи системного вызова `write`, чтение из файла — `read`.

Листинг 1.8 – Функция write и read

```
1 #include <fcntl.h>
2
3 ssize_t write(int fd, const void *buf, size_t size);
4 ssize_t read(int fd, void *buf, size_t count);
```

Функции `write` и `read` работают не с именем файла, а с файловым дескриптором. Однако для того, чтобы определить, можно ли выполнить операцию с файлом, необходимо знать его имя, так как файлы, для которых необходимо запретить те или иные операции, хранятся в виде списка имен этих файлов. Тогда следует перехватить функцию `open`, которая позволяет получить имя файла.

Системный вызов `open` осуществляет открытие файла.

Листинг 1.9 – Функция open

```
1 #include <fcntl.h>
2
3 int open(int dirfd, const char *pathname, int flags);
4 int open(int dirfd, const char *pathname, int flags, mode_t mode);
```

При перехвате `open`, если имя файла есть в списке, то необходимо сохранить идентификатор процесса, открывающего файл. Затем при перехвате `write` и `read` ориентироваться не на имя файла, а на идентификатор процесса и файловый дескриптор. Так можно будет однозначно определить, необходимо ли запретить чтение из файла или запись в него.

Таким образом, для того, чтобы запретить чтение из файла и запись в него, необходимо перехватить системные вызовы `open`, `write` и `read`. Перехват осуществляется при помощи создания указателей на системные вызовы.

Листинг 1.10 – Указатели на `open`, `write` и `read`

```
1 static asmlinkage long (*real_sys_open)(struct pt_regs *regs);  
2 static asmlinkage long (*real_sys_write)(struct pt_regs *regs);  
3 static asmlinkage long (*real_sys_read)(struct pt_regs *regs);
```

1.3 Вывод

В результате сравнительного анализа выбран способ перехвата функций ядра — `ftrace`, так как он позволяет перехватывать любые функции ядра и не требует его перекомпиляции. Для сокрытия файла необходимо перехватить функцию `getdents64`, для запрета чтения из файла и запись в файл — функции `open`, `read` и `write`, удаления — `unlink`.

2 Конструкторский раздел

2.1 IDEF0

На рисунке 2.1 приведена диаграмма состояний IDEF0 нулевого уровня, а на рисунке 2.2 — диаграмма состояний IDEF0 первого уровня.

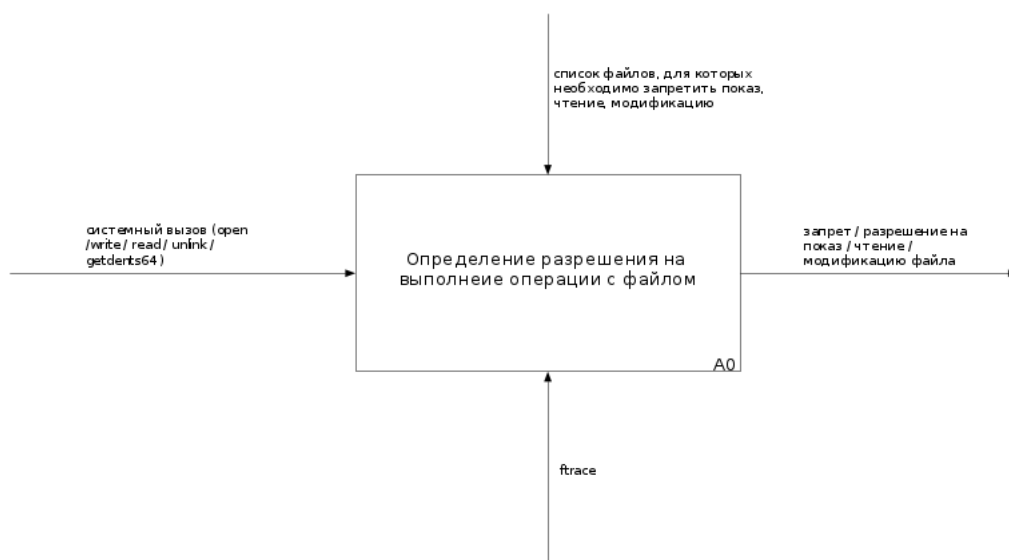


Рисунок 2.1 – Диаграмма состояний IDEF0 нулевого уровня

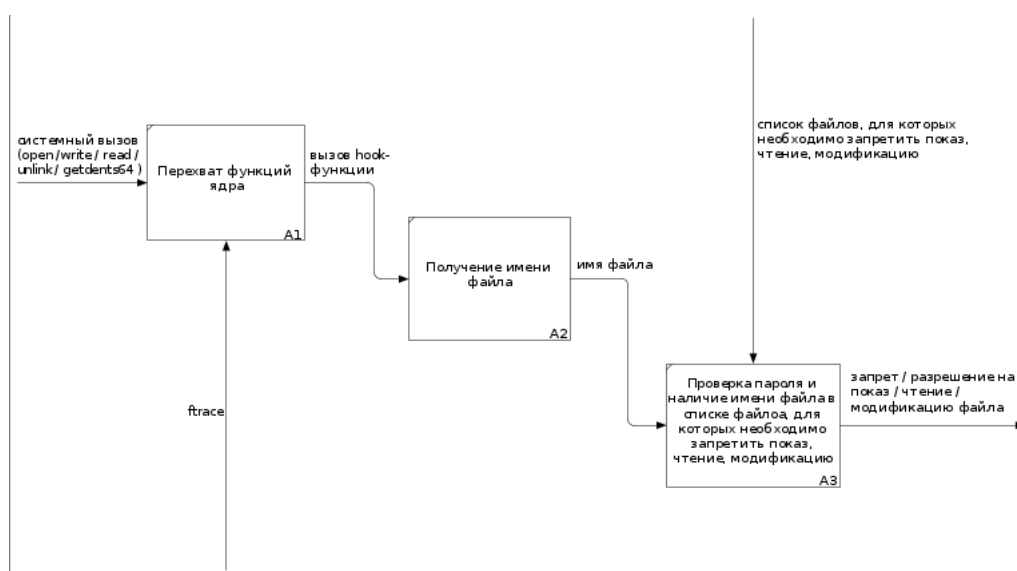


Рисунок 2.2 – Диаграмма состояний IDEF0 первого уровня

2.2 Алгоритм проверки необходимости сокрытия файла

На рисунке 2.3 приведена схема алгоритма проверки необходимости сокрытия файла.

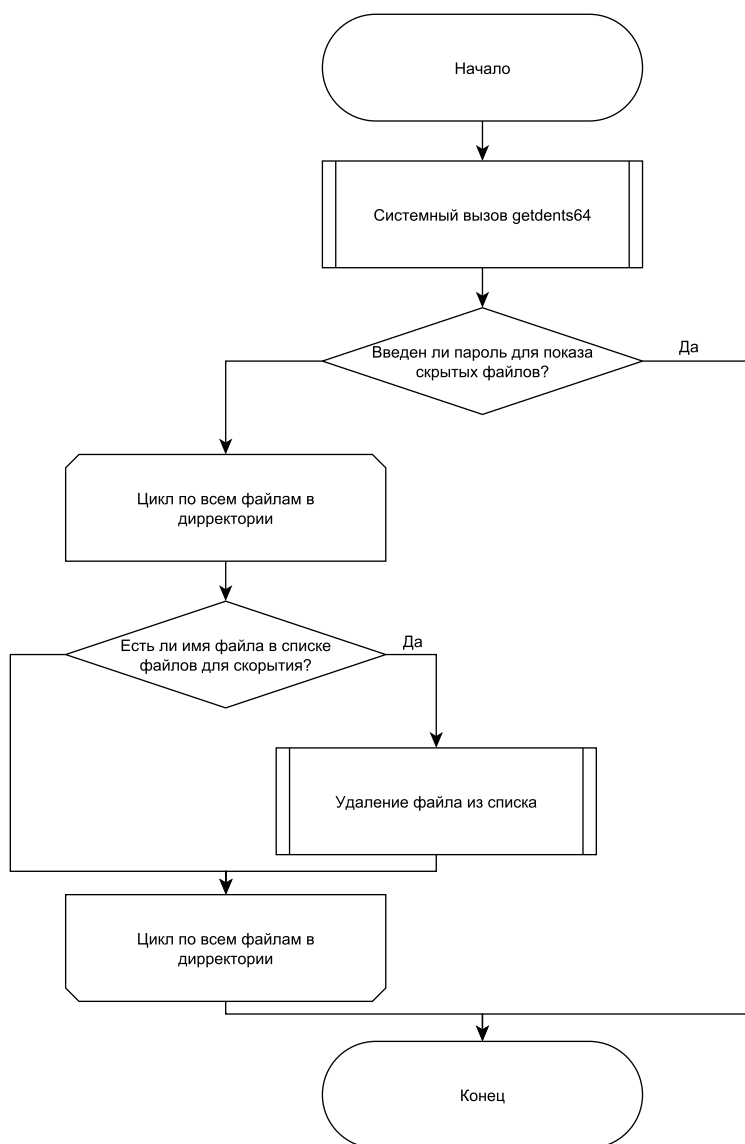


Рисунок 2.3 – Алгоритм проверки необходимости сокрытия файла

2.3 Алгоритм проверки разрешения на удаление файла

На рисунке 2.4 приведена схема алгоритма проверки разрешения на удаление файла.

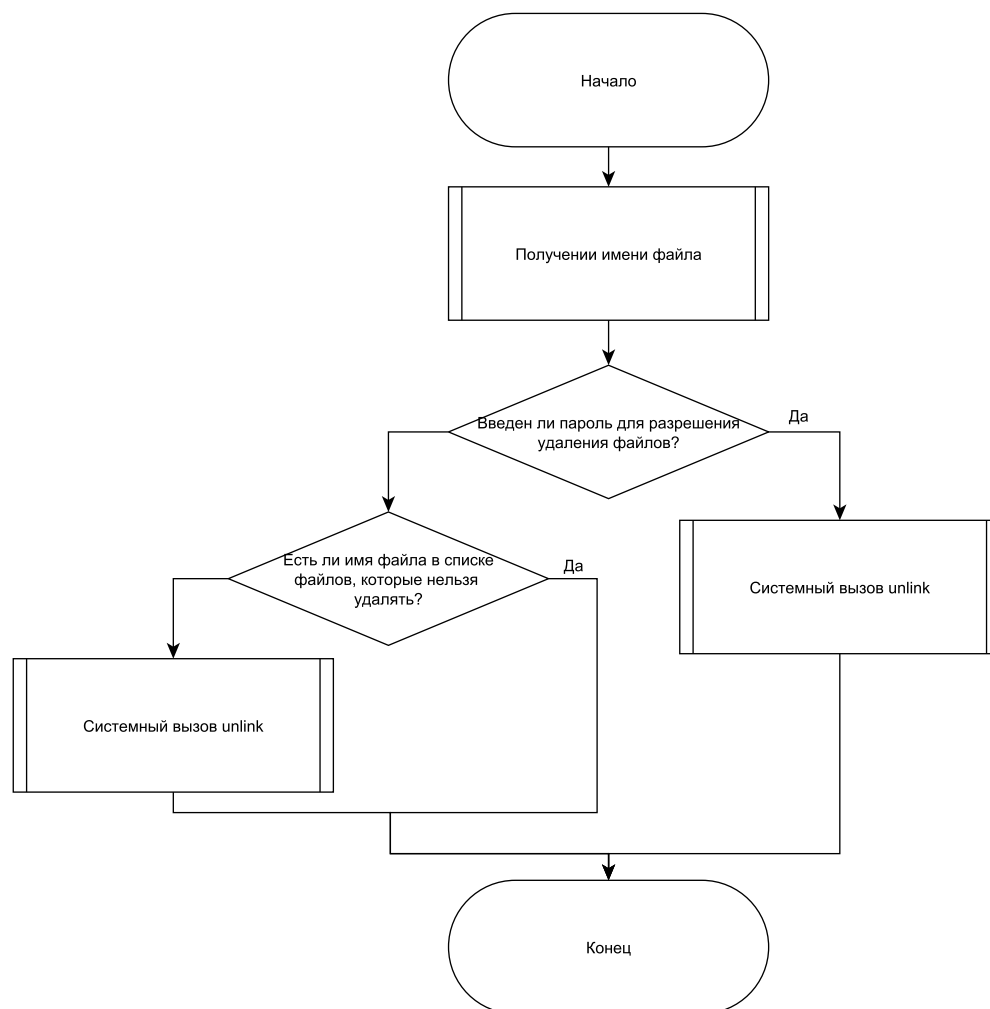


Рисунок 2.4 – Алгоритм проверки разрешения на удаление файла

2.4 Алгоритм проверки разрешения на запись в файл

На рисунке 2.5 приведена схема алгоритма проверки разрешения на запись в файл.

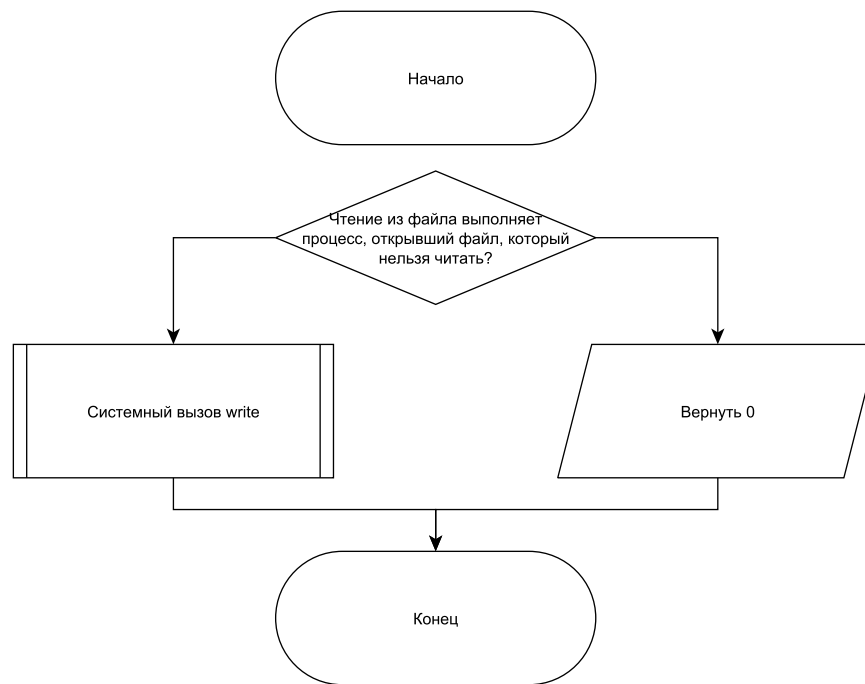


Рисунок 2.5 – Алгоритм проверки разрешения на удаление файла

2.5 Алгоритм проверки разрешения на чтение из файла

На рисунке 2.6 приведена схема алгоритма проверки разрешения на чтение из файла.

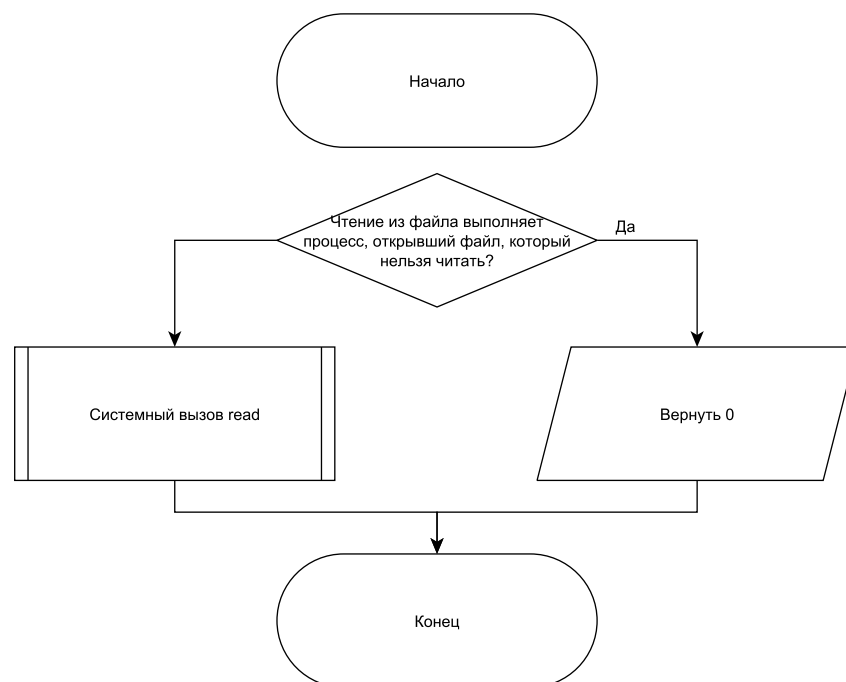


Рисунок 2.6 – Алгоритм проверки разрешения на удаление файла

3 Технологический раздел

3.1 Выбор языка и среды программирования

В качестве языка программирования был выбран язык Си. Для сборки модуля использовалась утилита make. В качестве среды программирования был выбран VSCode.

3.2 Реализация алгоритма проверки необходимости сокрытия файла

В листинге 3.1 приведена реализация алгоритма проверки необходимости сокрытия файла.

Листинг 3.1 – Реализация алгоритма проверки необходимости сокрытия файла

```
1 static asmlinkage int fh_sys_getdents64(const struct pt_regs
   *regs)
2 {
3     struct linux_dirent64 __user *dirent = (struct linux_dirent64
       *)regs->si;
4     struct linux_dirent64 *previous_dir = NULL, *current_dir,
       *dirent_ker = NULL;
5     unsigned long offset = 0;
6     int ret = real_sys_getdents64(regs);
7
8     if (ret <= 0)
9     {
10         return ret;
11     }
12
13     dirent_ker = kmalloc(ret, GFP_KERNEL);
14
15     if (dirent_ker == NULL)
16     {
17         return ret;
18     }
19
20     if (copy_from_user(dirent_ker, dirent, ret) != 0)
21     {
22         kfree(dirent_ker);
23         return ret;
```

```

24     }
25
26     while (offset < ret)
27     {
28         current_dir = (struct linux_dirent64 *)((char
                *)dirent_ker + offset);
29
30         if (check_fs_hidelist(current_dir->d_name))
31         {
32             if (current_dir == dirent_ker)
33             {
34                 ret -= current_dir->d_reclen;
35                 memmove(current_dir, (void *)current_dir +
36                     current_dir->d_reclen, ret);
37                 continue;
38             }
39             if (previous_dir)
40             {
41                 previous_dir->d_reclen += current_dir->d_reclen;
42             }
43         }
44         else
45         {
46             previous_dir = current_dir;
47         }
48         offset += current_dir->d_reclen;
49     }
50     if (copy_to_user(dirent, dirent_ker, ret) != 0)
51     {
52         kfree(dirent_ker);
53         return ret;
54     }
55     kfree(dirent_ker);
56     return ret;
57 }

```

3.3 Реализация алгоритма проверки разрешения на удаление файла

Реализация алгоритма проверки разрешения на удаление файла представлена в листинге 3.2.

Листинг 3.2 – Реализация алгоритма проверки разрешения на удаление файла

```
1 static asmlinkage long fh_sys_unlink(struct pt_regs *regs)
2 {
3     long ret = 0;
4     char *kernel_filename = get_filename((void*) regs->di);
5
6     if (!kernel_filename)
7     {
8         return real_sys_unlink(regs);
9     }
10
11     if (check_fs_blocklist(kernel_filename))
12     {
13         ret = 0;
14         kfree(kernel_filename);
15         return ret;
16     }
17
18     kfree(kernel_filename);
19     ret = real_sys_unlink(regs);
20
21     return ret;
22 }
```

3.4 Реализация алгоритма проверки разрешения на чтение из файла

Реализация алгоритма проверки разрешения на чтение из файла представлена в листинге 3.3.

Листинг 3.3 – Реализация алгоритма проверки разрешения на чтение из файла

```
1 static asmlinkage long fh_sys_read(struct pt_regs *regs)
2 {
3     long ret = 0;
4     struct task_struct *task = current;
5
6     if (task->pid == target_pid && regs->di == target_fd)
7     {
8         return 0;
9     }
10
```

```

11     ret = real_sys_read(regs);
12     return ret;
13 }

```

3.5 Реализация алгоритма проверки разрешения на запись в файл

Реализация алгоритма проверки разрешения на запись в файл представлена в листинге 3.4.

Листинг 3.4 – Реализация алгоритма проверки разрешения на запись в файл

```

1 static asmlinkage long fh_sys_write(struct pt_regs *regs)
2 {
3     long ret = 0;
4     struct task_struct *task;
5     task = current;
6
7     if (task->pid == target_pid && regs->di == target_fd)
8     {
9         return 0;
10    }
11
12    ret = real_sys_write(regs);
13    return ret;
14 }

```

3.6 Инициализация полей структуры ftrace_hook

Инициализация полей структуры ftrace_hook представлена в листинге 3.5.

Листинг 3.5 – Инициализация полей структуры ftrace_hook

```

1 static struct ftrace_hook demo_hooks[] = {
2     HOOK("write", fh_sys_write, &real_sys_write),
3     HOOK("read", fh_sys_read, &real_sys_read),
4     HOOK("open", fh_sys_open, &real_sys_open),
5     HOOK("unlink", fh_sys_unlink, &real_sys_unlink),
6     HOOK("getdents64", fh_sys_getdents64, &real_sys_getdents64)
7 };

```

3.7 Makefile

В листинге 3.6 представлен Makefile.

Листинг 3.6 – Makefile

```
1 CONFIG_MODULE_SIG=0
2 PWD := $(shell pwd)
3 KDIR_PATH ?= /lib/modules/$(shell uname -r)/build
4
5 obj-m += my_module.o
6 my_module-objs := main.o rootkit.o
7
8 ccflags-y := -Wall -Wno-unused-result -Wno-unused-function
9             -Wno-unused-variable
10
11 all:
12 $(MAKE) -C $(KDIR_PATH) M=$(PWD) modules
13
14 clean:
15 $(MAKE) -C $(KDIR_PATH) M=$(PWD) clean
```

Пример работы разработанного программного обеспечения

После загрузки модуля файлы из списков становятся невидимыми и защищёнными.

Листинг 3.7 – Добавление файлов в списки

```
1 artem@artem-pc:~$ echo hidden.txt > /proc/hidden
2 artem@artem-pc:~$ echo protected.txt > /proc/protected
```

Листинг 3.8 – Содержимое директории после загрузки модуля

```
1 artem@artem-pc:~/OS_CP$ ls
2 file.txt  protected.txt
```

Файл hidden.txt полностью исчез из листинга.

Листинг 3.9 – Отключение режима скрытия

```
1 artem@artem-pc:~$ echo 1234 > /dev/emblem
```

После ввода пароля 1234 файл снова виден:

Листинг 3.10 – Файлы снова видны

```
1 artem@artem-pc:~/OS_CP$ ls
2 file.txt  hidden.txt  protected.txt
```

Аналогично, попытка удалить или прочитать protected.txt ничего не даёт:

Листинг 3.11 – Защита от удаления и чтения работает

```
1 artem@artem-pc:~$ rm protected.txt
2 artem@artem-pc:~$ echo $?
3 1
4 artem@artem-pc:~$ cat protected.txt
5 (nothing appears)
```

Отключаем защиту:

Листинг 3.12 – Отключение режима защиты

```
1 artem@artem-pc:~$ echo 4321 > /dev/emblem
```

Теперь операции проходят успешно:

Листинг 3.13 – Файл снова доступен

```
1 artem@artem-pc:~$ cat protected.txt
2 This is a protected file
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был определен способ перехвата системных вызовов — путем регистрации функций перехвата с использованием `ftrace`, так как он позволяет перехватывать любые функции ядра и не требует его перекомпиляции.

Для сокрытия файла была перехвачена функция `getdents64`, для запрета чтения из файла и записи в файл — функции `open`, `read` и `write`, удаления — `unlink`.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Стивенс Р., Раго С.* UNIX. Профессиональное программирование. — СПб. : Питер, 2018. — 944 с.
2. Код ядра Linux. — URL: <https://elixir.bootlin.com/linux/latest/source> (дата обращения: 10.11.2025).
3. Встраивание в ядро Linux: перехват функций. — URL: <https://habr.com/ru/companies/securitycode/articles/237089/> (дата обращения: 12.11.2025).

ПРИЛОЖЕНИЕ А

Листинг 3.1 – Файл main.c

```
1 #include <linux/module.h>
2 #include <linux/kernel.h>
3 #include <linux/init.h>
4 #include <linux/uaccess.h>
5 #include <linux/fs.h>
6 #include <linux/device.h>
7 #include <linux/cdev.h>
8 #include <linux/proc_fs.h>
9 #include <linux/string.h>
10
11 #include "rootkit.h"
12
13 MODULE_DESCRIPTION("fs_module");
14 MODULE_AUTHOR("Novikov Artem");
15 MODULE_LICENSE("GPL");
16
17 #define PROC_FILE_NAME_HIDDEN "hidden"
18 #define PROC_FILE_NAME_PROTECTED "protected"
19
20 static char buffer[MAX_BUF_SIZE];
21 char tmp_buffer[MAX_BUF_SIZE];
22 char hidden_files[100][100];
23 int hidden_index = 0;
24 char protected_files[100][100];
25 int protected_index = 0;
26
27 static ssize_t my_proc_write(struct file *file, const char __user
    *buf, size_t len, loff_t *ppos)
28 {
29     DBG("my_proc_write called");
30
31     if (len > MAX_BUF_SIZE - write_index + 1)
32     {
33         DBG("buffer overflow");
34         return -ENOMEM;
35     }
36
```

```

37     if (copy_from_user(&buffer[write_index], buf, len) != 0)
38     {
39         DBG("copy_from_user fail");
40         return -EFAULT;
41     }
42
43     write_index += len;
44     buffer[write_index - 1] = '\0';
45
46     if (strcmp(file->f_path.dentry->d_name.name,
47         PROC_FILE_NAME_HIDDEN) == 0)
48     {
49         snprintf(hidden_files[hidden_index], len, "%s",
50             &buffer[write_index - len]);
51         hidden_index++;
52         DBG("file written to hidden %s",
53             hidden_files[hidden_index - 1]);
54     }
55     else if (strcmp(file->f_path.dentry->d_name.name,
56         PROC_FILE_NAME_PROTECTED) == 0)
57     {
58         snprintf(protected_files[protected_index], len, "%s",
59             &buffer[write_index - len]);
60         protected_index++;
61         DBG("file written to protected %s",
62             protected_files[protected_index - 1]);
63     }
64     else
65     {
66         DBG("Unknown file->f_path.dentry->d_name.name");
67     }
68     return len;
69 }
70
71 static ssize_t my_proc_read(struct file *file, char __user *buf,
72     size_t len, loff_t *f_pos)
73 {
74     DBG("my_proc_read called");
75
76     if (*f_pos > 0 || write_index == 0)
77     return 0;

```

```

71
72     if (read_index >= write_index)
73         read_index = 0;
74
75     int read_len = snprintf(tmp_buffer, MAX_BUF_SIZE, "%s\n",
76                             &buffer[read_index]);
77
78     if (copy_to_user(buf, tmp_buffer, read_len) != 0)
79     {
80         DBG("copy_to_user error.\n");
81         return -EFAULT;
82     }
83
84     read_index += read_len;
85     *f_pos += read_len;
86
87     return read_len;
88 }
89
90 static const struct proc_ops fops = {
91     .proc_read = my_proc_read,
92     .proc_write = my_proc_write,
93 };
94
95 static int __init fh_init(void)
96 {
97     struct device *fake_device;
98     int error = 0;
99     dev_t dev = 0;
100
101     proc_file_hidden = proc_create(PROC_FILE_NAME_HIDDEN, S_IRUGO
102     | S_IWUGO, NULL, &fops);
103     if (!proc_file_hidden)
104     {
105         DBG("call proc_create() fail");
106         return -ENOMEM;
107     }
108
109     proc_file_protected = proc_create(PROC_FILE_NAME_PROTECTED,
110     S_IRUGO | S_IWUGO, NULL, &fops);
111     if (!proc_file_protected)

```

```

109     {
110         remove_proc_entry(PROC_FILE_NAME_HIDDEN, NULL);
111         DBG("call proc_create() fail");
112         return -ENOMEM;
113     }
114     DBG("proc file created");
115
116     error = init_kallsyms();
117     if (error) {
118         remove_proc_entry(PROC_FILE_NAME_HIDDEN, NULL);
119         remove_proc_entry(PROC_FILE_NAME_PROTECTED, NULL);
120         DBG("Failed to init kallsyms: %d", error);
121         return error;
122     }
123
124     error = start_hook_resources();
125     if (error)
126     {
127         remove_proc_entry(PROC_FILE_NAME_HIDDEN, NULL);
128         remove_proc_entry(PROC_FILE_NAME_PROTECTED, NULL);
129         DBG("return in hook functions");
130         return error;
131     }
132
133     error = alloc_chrdev_region(&dev, 0, 1, "table");
134     if (error < 0)
135     {
136         remove_proc_entry(PROC_FILE_NAME_HIDDEN, NULL);
137         remove_proc_entry(PROC_FILE_NAME_PROTECTED, NULL);
138         return error;
139     }
140
141     major = MAJOR(dev);
142     minor = MINOR(dev);
143
144     fake_class = class_create("custom_char_class");
145     if (IS_ERR(fake_class)) {
146         remove_proc_entry(PROC_FILE_NAME_HIDDEN, NULL);
147         remove_proc_entry(PROC_FILE_NAME_PROTECTED, NULL);
148         unregister_chrdev_region(dev, 1);
149         return PTR_ERR(fake_class);

```

```

150     }
151
152     cdev_init(&fake_cdev, &fake_fops);
153     fake_cdev.owner = THIS_MODULE;
154     cdev_add(&fake_cdev, dev, 1);
155
156     fake_device = device_create(fake_class,
157     NULL,
158     dev,
159     NULL,
160     "emblem");
161
162     if (IS_ERR(fake_device))
163     {
164         remove_proc_entry(PROC_FILE_NAME_HIDDEN, NULL);
165         remove_proc_entry(PROC_FILE_NAME_PROTECTED, NULL);
166         class_destroy(fake_class);
167         unregister_chrdev_region(dev, 1);
168         return -1;
169     }
170
171     return 0;
172 }
173
174 static void __exit fh_exit(void)
175 {
176     if (proc_file_hidden)
177     {
178         remove_proc_entry(PROC_FILE_NAME_HIDDEN, NULL);
179         DBG("proc file removed (hidden)");
180     }
181
182     if (proc_file_protected)
183     {
184         remove_proc_entry(PROC_FILE_NAME_PROTECTED, NULL);
185         DBG("proc file removed (protected)");
186     }
187
188     fh_remove_hooks(demo_hooks, ARRAY_SIZE(demo_hooks));
189     unregister_chrdev_region(MKDEV(major, 0), 1);
190     device_destroy(fake_class, MKDEV(major, 0));

```

```

191     cdev_del(&fake_cdev);
192     class_destroy(fake_class);
193 }
194
195 module_init(fh_init);
196 module_exit(fh_exit);

```

Листинг 3.2 – Файл rootkit.h

```

1 #ifndef ROOTKIT_H
2 #define ROOTKIT_H
3
4 #include <linux/ftrace.h>
5 #include <linux/kallsyms.h>
6 #include <linux/syscalls.h>
7 #include <linux/kernel.h>
8 #include <linux/version.h>
9 #include <linux/delay.h>
10 #include <linux/kthread.h>
11 #include <asm/ptrace.h>
12 #include <linux/fcntl.h>
13 #include <linux/types.h>
14 #include <linux/dirent.h>
15 #include <linux/device.h>
16 #include <linux/cdev.h>
17 #include <linux/module.h>
18 #include <linux/init.h>
19 #include <linux/fs.h>
20 #include <linux/proc_fs.h>
21 #include <linux/kprobes.h>
22 #include <linux/string.h>
23
24 #define FILE_NAME (strchr(__FILE__, '/') ? strchr(__FILE__,
    '/') + 1 : __FILE__)
25
26 #define DBG(fmt, ...) \
27     printk(KERN_INFO "DBG: %s(%d): " fmt "\n", FILE_NAME, __LINE__,
    ##__VA_ARGS__)
28
29 #define MAX_BUF_SIZE 1000
30
31 static struct proc_dir_entry *proc_file_hidden;
32 static struct proc_dir_entry *proc_file_protected;

```

```

33
34 extern char hidden_files[100][100];
35 extern int hidden_index;
36 extern char protected_files[100][100];
37 extern int protected_index;
38
39 static int read_index = 0;
40 static int write_index = 0;
41 static unsigned int major;
42 static unsigned int minor;
43 static struct class *fake_class;
44 static struct cdev fake_cdev;
45 static short fs_hidden = 1;
46 static short fs_protect = 1;
47
48 ssize_t fake_write(struct file *file, const char __user *buf,
49                    size_t count, loff_t *offset);
50
51 static struct file_operations fake_fops = {
52     .write = fake_write,
53 };
54
55 int check_fs_blocklist(char *input);
56 int check_fs_hidelist(char *input);
57
58 static unsigned int target_fd = 0;
59 static unsigned int target_pid = 0;
60
61 typedef unsigned long (*kallsyms_lookup_name_t)(const char *name);
62 extern kallsyms_lookup_name_t kallsyms_lookup_name_ptr;
63
64 int init_kallsyms(void);
65
66 static unsigned long lookup_name(const char *name)
67 {
68     if (!kallsyms_lookup_name_ptr) {
69         DBG("kallsyms_lookup_name_ptr not initialized");
70         return 0;
71     }
72     return kallsyms_lookup_name_ptr(name);
73 }

```

```

73
74 struct ftrace_hook {
75     const char *name;
76     void *function;
77     void *original;
78
79     unsigned long address;
80     struct ftrace_ops ops;
81 };
82
83 #ifndef MCOUNT_INSN_SIZE
84 #define MCOUNT_INSN_SIZE 5
85 #endif
86
87 static int fh_resolve_hook_address(struct ftrace_hook *hook)
88 {
89     hook->address = lookup_name(hook->name);
90
91     if (!hook->address) {
92         DBG("unresolved symbol: %s, trying alternatives\n",
93             hook->name);
94         // Попробуем альтернативные имена
95         char alt_name[128];
96         // Попробуем __do_sys_ * вместо __x64_sys_ *
97         if (strncmp(hook->name, "__x64_sys_", 10) == 0) {
98             snprintf(alt_name, sizeof(alt_name), "__do_sys_%s",
99                 hook->name + 10);
100             hook->address = lookup_name(alt_name);
101         }
102         // Попробуем sys_ * без префикса
103         if (!hook->address && strncmp(hook->name, "__x64_sys_",
104             10) == 0) {
105             snprintf(alt_name, sizeof(alt_name), "sys_%s",
106                 hook->name + 10);
107             hook->address = lookup_name(alt_name);
108         }
109         if (!hook->address) {
110             DBG("unresolved symbol: %s (all alternatives
111                 failed)\n", hook->name);
112             return -ENOENT;
113         }
114     }

```



```

109         DBG("resolved via alternative: %s -> %lx\n", hook->name,
              hook->address);
110     }
111     *((unsigned long*) hook->original) = hook->address +
        MCOUNT_INSN_SIZE;
112
113     return 0;
114 }
115
116 static void notrace fh_ftrace_thunk(unsigned long ip, unsigned
        long parent_ip,
117 struct ftrace_ops *ops, struct ftrace_regs *fregs)
118 {
119     struct pt_regs *regs;
120     struct ftrace_hook *hook = container_of(ops, struct
        ftrace_hook, ops);
121
122     regs = ftrace_get_regs(fregs);
123     if (!regs)
124         return;
125
126     regs->ip = (unsigned long)hook->function;
127 }
128
129 int fh_install_hook(struct ftrace_hook *hook);
130 void fh_remove_hook(struct ftrace_hook *hook);
131 int fh_install_hooks(struct ftrace_hook *hooks, size_t count);
132 void fh_remove_hooks(struct ftrace_hook *hooks, size_t count);
133
134 static char *get_filename(const char __user *filename)
135 {
136     char *kernel_filename = NULL;
137     kernel_filename = kmalloc(4096, GFP_KERNEL);
138     if (!kernel_filename)
139         return NULL;
140
141     if (strncpy_from_user(kernel_filename, filename, 4096) < 0) {
142         kfree(kernel_filename);
143         return NULL;
144     }
145     return kernel_filename;

```

```

146 }
147
148 static asmlinkage long (*real_sys_write)(struct pt_regs *regs);
149 static asmlinkage long fh_sys_write(struct pt_regs *regs)
150 {
151     long ret = 0;
152     struct task_struct *task;
153     task = current;
154
155     if (task->pid == target_pid && regs->di == target_fd)
156     {
157         return 0;
158     }
159
160     ret = real_sys_write(regs);
161     return ret;
162 }
163
164 static asmlinkage long (*real_sys_read)(struct pt_regs *regs);
165 static asmlinkage long fh_sys_read(struct pt_regs *regs)
166 {
167     long ret = 0;
168     struct task_struct *task = current;
169
170     if (task->pid == target_pid && regs->di == target_fd)
171     {
172         return 0;
173     }
174
175     ret = real_sys_read(regs);
176     return ret;
177 }
178
179 static asmlinkage long (*real_sys_open)(struct pt_regs *regs);
180 static asmlinkage long fh_sys_open(struct pt_regs *regs)
181 {
182     long ret;
183     char *kernel_filename;
184     struct task_struct *task;
185     task = current;
186     kernel_filename = get_filename((void*) regs->di);

```

```

187
188     if (!kernel_filename)
189     {
190         return real_sys_open(regs);
191     }
192
193     if (check_fs_blocklist(kernel_filename))
194     {
195         DBG("our file is opened by process with id: %d",
196             task->pid);
197         DBG("opened file : '%s'", kernel_filename);
198         kfree(kernel_filename);
199         ret = real_sys_open(regs);
200         DBG("fd returned is %ld", ret);
201         if (ret >= 0)
202         {
203             target_fd = ret;
204             target_pid = task->pid;
205         }
206         return ret;
207     }
208
209     kfree(kernel_filename);
210     ret = real_sys_open(regs);
211
212     return ret;
213 }
214
215 static asmlinkage long (*real_sys_unlink)(struct pt_regs *regs);
216 static asmlinkage long fh_sys_unlink(struct pt_regs *regs)
217 {
218     long ret = 0;
219     char *kernel_filename = get_filename((void*) regs->di);
220
221     if (!kernel_filename)
222     {
223         return real_sys_unlink(regs);
224     }
225
226     if (check_fs_blocklist(kernel_filename))
227     {

```

```

227         ret = 0;
228         kfree(kernel_filename);
229         return ret;
230     }
231
232     kfree(kernel_filename);
233     ret = real_sys_unlink(regs);
234
235     return ret;
236 }
237
238 static asmlinkage long (*real_sys_getdents64)(const struct
        pt_regs *);
239 static asmlinkage int fh_sys_getdents64(const struct pt_regs
        *regs)
240 {
241     struct linux_dirent64 __user *dirent = (struct linux_dirent64
        *)regs->si;
242     struct linux_dirent64 *previous_dir = NULL, *current_dir,
        *dirent_ker = NULL;
243     unsigned long offset = 0;
244     int ret = real_sys_getdents64(regs);
245
246     if (ret <= 0)
247     {
248         return ret;
249     }
250
251     dirent_ker = kmalloc(ret, GFP_KERNEL);
252
253     if (dirent_ker == NULL)
254     {
255         return ret;
256     }
257
258     if (copy_from_user(dirent_ker, dirent, ret) != 0)
259     {
260         kfree(dirent_ker);
261         return ret;
262     }
263

```

```

264 while (offset < ret)
265 {
266     current_dir = (struct linux_dirent64 *)((char
                *)dirent_ker + offset);
267
268     if (check_fs_hidelist(current_dir->d_name))
269     {
270         if (current_dir == dirent_ker)
271         {
272             ret -= current_dir->d_reclen;
273             memmove(current_dir, (void *)current_dir +
274                     current_dir->d_reclen, ret);
275             continue;
276         }
277         if (previous_dir)
278         {
279             previous_dir->d_reclen += current_dir->d_reclen;
280         }
281     }
282     else
283     {
284         previous_dir = current_dir;
285     }
286     offset += current_dir->d_reclen;
287 }
288 if (copy_to_user(dirent, dirent_ker, ret) != 0)
289 {
290     kfree(dirent_ker);
291     return ret;
292 }
293 kfree(dirent_ker);
294 return ret;
295 }
296
297 #define SYSCALL_NAME(name) ("__x64_sys_" name)
298
299 #define HOOK(_name, _function, _original) \
300 { \
301     .name = SYSCALL_NAME(_name), \
302     .function = (_function), \
303     .original = (_original), \

```

```

304 }
305
306 static struct ftrace_hook demo_hooks[] = {
307     HOOK("write", fh_sys_write, &real_sys_write),
308     HOOK("read", fh_sys_read, &real_sys_read),
309     HOOK("open", fh_sys_open, &real_sys_open),
310     HOOK("unlink", fh_sys_unlink, &real_sys_unlink),
311     HOOK("getdents64", fh_sys_getdents64, &real_sys_getdents64)
312 };
313
314 static int start_hook_resources(void)
315 {
316     int err;
317     err = fh_install_hooks(demo_hooks, ARRAY_SIZE(demo_hooks));
318     if (err)
319     {
320         return err;
321     }
322     return 0;
323 }
324
325 #endif

```

ЛИСТИНГ 3.3 – Файл rootkit.c

```

1 #include "rootkit.h"
2
3 kallsyms_lookup_name_t kallsyms_lookup_name_ptr = NULL;
4
5 int init_kallsyms(void)
6 {
7     struct kprobe kp = {
8         .symbol_name = "kallsyms_lookup_name"
9     };
10    int ret;
11    unsigned long addr;
12
13    ret = register_kprobe(&kp);
14    if (ret < 0) {
15        DBG("register_kprobe failed for kallsyms_lookup_name: %d", ret);
16        return ret;
17    }
18

```

```

19 addr = (unsigned long)kp.addr;
20 unregister_kprobe(&kp);
21
22 if (!addr) {
23     DBG("Failed to resolve kallsyms_lookup_name address");
24     return -ENOENT;
25 }
26
27 kallsyms_lookup_name_ptr = (kallsyms_lookup_name_t)addr;
28 DBG("kallsyms_lookup_name resolved at %lx", addr);
29 DBG("kallsyms_lookup_name_ptr set to %px", (void
    *)kallsyms_lookup_name_ptr);
30 return 0;
31 }
32
33 ssize_t fake_write(struct file *flip, const char __user *buf,
    size_t count, loff_t *offset)
34 {
35     char message[128];
36     memset(message, 0, 127);
37
38     if (copy_from_user(message, buf, 127) != 0)
39     {
40         return -EFAULT;
41     }
42
43     if (strstr(message, "1234") != NULL)
44     {
45         fs_hidden = fs_hidden ? 0 : 1;
46     }
47
48     if (strstr(message, "4321") != NULL)
49     {
50         fs_protect = fs_protect ? 0 : 1;
51     }
52
53     return count;
54 }
55
56 int check_fs_blocklist(char *input)
57 {

```

```

58 int i = 0;
59
60 if (fs_protect == 0)
61 {
62 return 0;
63 }
64
65 if (strlen(protected_files[0]) <= 2)
66 {
67 return 0;
68 }
69
70 while (i != protected_index)
71 {
72 if (strstr(input, protected_files[i]) != NULL)
73 return 1;
74 i++;
75 }
76
77 return 0;
78 }
79
80 int check_fs_hidelist(char *input)
81 {
82 int i = 0;
83
84 if (fs_hidden == 0)
85 {
86 return 0;
87 }
88
89 if (strlen(hidden_files[0]) <= 2)
90 {
91 return 0;
92 }
93
94 while (i != hidden_index)
95 {
96 if (strstr(input, hidden_files[i]) != NULL)
97 return 1;
98 i++;

```



```

99 }
100
101 return 0;
102 }
103
104 int fh_install_hook(struct ftrace_hook *hook)
105 {
106     int error;
107
108     error = fh_resolve_hook_address(hook);
109     if (error)
110     {
111         return error;
112     }
113
114     hook->ops.func = fh_ftrace_thunk;
115     hook->ops.flags = FTRACE_OPS_FL_SAVE_REGS |
116     FTRACE_OPS_FL_IPMODIFY |
117     FTRACE_OPS_FL_RECURSION;
118
119     error = ftrace_set_filter_ip(&hook->ops, hook->address, 0, 0);
120     if (error)
121     {
122         DBG("ftrace_set_filter_ip() failed: %d\n", error);
123         return error;
124     }
125
126     error = register_ftrace_function(&hook->ops);
127     if (error)
128     {
129         DBG("register_ftrace_function() failed: %d\n", error);
130         ftrace_set_filter_ip(&hook->ops, hook->address, 1, 0);
131         return error;
132     }
133     return 0;
134 }
135
136 void fh_remove_hook(struct ftrace_hook *hook)
137 {
138     int err;
139

```

```

140 err = unregister_ftrace_function(&hook->ops);
141 if (err)
142 {
143 DBG("unregister_ftrace_function() failed: %d\n", err);
144 }
145
146 err = ftrace_set_filter_ip(&hook->ops, hook->address, 1, 0);
147 if (err)
148 {
149 DBG("ftrace_set_filter_ip() failed: %d\n", err);
150 }
151 }
152
153 int fh_install_hooks(struct ftrace_hook *hooks, size_t count)
154 {
155 int err;
156 size_t i;
157
158 for (i = 0; i < count; i++) {
159 err = fh_install_hook(&hooks[i]);
160 if (err)
161 {
162 while (i != 0)
163 {
164 fh_remove_hook(&hooks[--i]);
165 }
166 return err;
167 }
168 }
169 return 0;
170 }
171
172 void fh_remove_hooks(struct ftrace_hook *hooks, size_t count)
173 {
174 size_t i;
175
176 for (i = 0; i < count; i++)
177 {
178 fh_remove_hook(&hooks[i]);
179 }
180 } #include "rootkit.h"

```

```

181
182 kallsyms_lookup_name_t kallsyms_lookup_name_ptr = NULL;
183
184 int init_kallsyms(void)
185 {
186     struct kprobe kp = {
187         .symbol_name = "kallsyms_lookup_name"
188     };
189     int ret;
190     unsigned long addr;
191
192     ret = register_kprobe(&kp);
193     if (ret < 0) {
194         DBG("register_kprobe failed for kallsyms_lookup_name:
195             %d", ret);
196         return ret;
197     }
198
199     addr = (unsigned long)kp.addr;
200     unregister_kprobe(&kp);
201
202     if (!addr) {
203         DBG("Failed to resolve kallsyms_lookup_name address");
204         return -ENOENT;
205     }
206
207     kallsyms_lookup_name_ptr = (kallsyms_lookup_name_t)addr;
208     DBG("kallsyms_lookup_name resolved at %lx", addr);
209     DBG("kallsyms_lookup_name_ptr set to %px", (void
210         *)kallsyms_lookup_name_ptr);
211     return 0;
212 }
213
214 ssize_t fake_write(struct file *flip, const char __user *buf,
215     size_t count, loff_t *offset)
216 {
217     char message[128];
218     memset(message, 0, 127);
219
220     if (copy_from_user(message, buf, 127) != 0)
221     {

```

```

219         return -EFAULT;
220     }
221
222     if (strstr(message, "1234") != NULL)
223     {
224         fs_hidden = fs_hidden ? 0 : 1;
225     }
226
227     if (strstr(message, "4321") != NULL)
228     {
229         fs_protect = fs_protect ? 0 : 1;
230     }
231
232     return count;
233 }
234
235 int check_fs_blocklist(char *input)
236 {
237     int i = 0;
238
239     if (fs_protect == 0)
240     {
241         return 0;
242     }
243
244     if (strlen(protected_files[0]) <= 2)
245     {
246         return 0;
247     }
248
249     while (i != protected_index)
250     {
251         if (strstr(input, protected_files[i]) != NULL)
252             return 1;
253         i++;
254     }
255
256     return 0;
257 }
258
259 int check_fs_hidelist(char *input)

```

```

260 {
261     int i = 0;
262
263     if (fs_hidden == 0)
264     {
265         return 0;
266     }
267
268     if (strlen(hidden_files[0]) <= 2)
269     {
270         return 0;
271     }
272
273     while (i != hidden_index)
274     {
275         if (strstr(input, hidden_files[i]) != NULL)
276             return 1;
277         i++;
278     }
279
280     return 0;
281 }
282
283 int fh_install_hook(struct ftrace_hook *hook)
284 {
285     int error;
286
287     error = fh_resolve_hook_address(hook);
288     if (error)
289     {
290         return error;
291     }
292
293     hook->ops.func = fh_ftrace_thunk;
294     hook->ops.flags = FTRACE_OPS_FL_SAVE_REGS |
295     FTRACE_OPS_FL_IPMODIFY |
296     FTRACE_OPS_FL_RECURSION;
297
298     error = ftrace_set_filter_ip(&hook->ops, hook->address, 0, 0);
299     if (error)
300     {

```

```

301         DBG("ftrace_set_filter_ip() failed: %d\n", error);
302         return error;
303     }
304
305     error = register_ftrace_function(&hook->ops);
306     if (error)
307     {
308         DBG("register_ftrace_function() failed: %d\n", error);
309         ftrace_set_filter_ip(&hook->ops, hook->address, 1, 0);
310         return error;
311     }
312     return 0;
313 }
314
315 void fh_remove_hook(struct ftrace_hook *hook)
316 {
317     int err;
318
319     err = unregister_ftrace_function(&hook->ops);
320     if (err)
321     {
322         DBG("unregister_ftrace_function() failed: %d\n", err);
323     }
324
325     err = ftrace_set_filter_ip(&hook->ops, hook->address, 1, 0);
326     if (err)
327     {
328         DBG("ftrace_set_filter_ip() failed: %d\n", err);
329     }
330 }
331
332 int fh_install_hooks(struct ftrace_hook *hooks, size_t count)
333 {
334     int err;
335     size_t i;
336
337     for (i = 0; i < count; i++) {
338         err = fh_install_hook(&hooks[i]);
339         if (err)
340         {
341             while (i != 0)

```

```
342         {
343             fh_remove_hook(&hooks[--i]);
344         }
345         return err;
346     }
347 }
348 return 0;
349 }
350
351 void fh_remove_hooks(struct ftrace_hook *hooks, size_t count)
352 {
353     size_t i;
354
355     for (i = 0; i < count; i++)
356     {
357         fh_remove_hook(&hooks[i]);
358     }
359 }
```